



**JOMO KENYATTA UNIVERSITY
OF
AGRICULTURE & TECHNOLOGY**

SCHOOL OF OPEN, DISTANCE AND eLEARNING

P.O. Box 62000, 00200

Nairobi, Kenya

E-mail: elearning@jkuat.ac.ke

BIT 2114 Object Oriented Analysis and Design

LAST REVISION ON April 28, 2014





This presentation is intended to be covered within one week. The notes, examples and exercises should be supplemented with a good textbook. Most of the exercises have solutions/answers appearing elsewhere and accessible by clicking the green **Exercise** tag. To move back to the same page click the same tag appearing at the end of the solution/answer.

Errors and omissions in these notes are entirely the responsibility of the author who should only be contacted through the **Department of Curricula & Delivery (SODeL)** and suggested corrections may be e-mailed to elarning@jkuat.ac.ke.



LESSON 1

Object-Oriented Analysis and Design

Learning outcomes

Upon completing this topic, you should be able to:

1. Define the terms used in object orientation.
2. Outline the importance of object oriented analysis and design phase.
3. Describe attributes of objects.
4. Explain usefulness of Object oriented concepts such as abstraction, modularity and Inheritance.



1.1. Introduction

1.1.1. Review of Approaches to System Development

There are many ways to develop an information system. Developers rely on many concepts for help, including methodologies, models, tools, and techniques.

Methodologies, Models, Tools, and Techniques

1. System Development Methodology

- A system development methodology Provides guidelines to follow for completing every activity in the system development life cycle, including specific models, tools, techniques, and a system development life cycle.
- There are many kind of methodologies



1. Models

- A representation of some important aspect of the real world
- Some models used in system development
 - Flow chart
 - Data flow diagram (DFD)
 - Entity-relationship diagram (ERD)
 - Structure chart
 - Use case diagram
 - Class diagram
 - Sequence diagram
- Some models used to manage the development process
 - PERT chart



- Gantt chart
- Organizational hierarchy chart
- Financial analysis models - NPV, ROI

1. Tools

- Software support that helps create models or other components required in the project.
- Some tools used in system development
 - Project management application
 - Drawing/graphics application
 - Word processor/text editor
 - CASE tools (Computer-aided system engineering tool designed to help a systems analyst complete development tasks)
 - Integrated development environment (IDE)



- Database management application
- Reverse-engineering tool
- Code generator tool

1. Techniques

- A collection of guidelines that help the analyst complete a system development activity or task.
- Some techniques used in system development
 - Strategic planning techniques
 - Project management techniques
 - User interviewing techniques
 - Data-modeling techniques
 - Relational database design techniques
 - Structured analysis technique
 - Structured design technique



BIT 2114 Object-Oriented Analysis and Design

- Structured programming techniques
- Software-testing techniques
- Object-oriented analysis and design techniques

Relationship Between Components of a Methodology

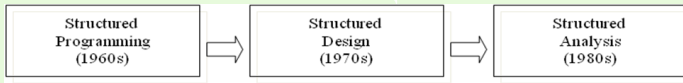


Figure 1.1: Relationship between Methodology, Techniques and Models

1.1.2. The Evolution of Systems Development Methodologies

1. Systems Development Life Cycle
2. The Structured Analysis and Design Approach



3. The Information Engineering Approach
4. The Object-Oriented Approach

Systems Development Life Cycle (SDLC)

- Considers systems development as a single cycle of sequential steps
- There might be other cycles during the operation/maintenance of the system
- Many variations of SDLC in practice
 - Based on variation of names for phases
 - No matter which one, activities/tasks are similar
- Some increase emphasis on people
 - User-centered design, participatory design



BIT 2114 Object-Oriented Analysis and Design

- Sociotechnical systems
- Some increase speed of development
 - Rapid application development (RAD)
 - Prototyping
- Advantages
 - Emphasis on planning and analysis which forces analysts to spend a significant amount of time on analysis, thereby understanding the problem thoroughly before designing and coding the system.
 - User requirements are determined and documented in detail before embarking on the risk of design and implementation
- Limitations



BIT 2114 Object-Oriented Analysis and Design

- Users might not be able to articulate requirements accurately
- User requirements might change as time passes, and is usually difficult to revisit early steps in the processes as necessitated by the changes.

The Structured Approach

System development using structured programming, structure analysis, and structured design techniques. Sometime referred to as structure analysis and design techniques (SADT).

Structured Programming

- Structured Program
 - A program or program module that has one begin-



ning and one ending, and each step in the program execution consists of sequence, selection, or repetition constructs.

- Top-down Programming (Modular Programming)
 - Divides complex programs into a hierarchy of program modules, the top module calls the low level modules to accomplish the task.
- Structured Design
 - A technique providing guidelines for deciding what the set of programs should be, what each program should accomplish, and how the programs should be organized into a hierarchy
 - Two main principles



- * Modules should be designed so that they are (1) Loosely coupled: each module is as independent of other modules as possible. Later modification will not affect other modules. (2) Highly cohesive: each module accomplishes one clear task. Easy to understand, change of a module will not affect other modules.

– Structured design's quality is defined in terms of how easily the design can be understood and modified later.

– Later developments of structured design

- * File and database design techniques to be used with Structured Design
- * User interface design techniques to be used with



Structured Design

- Structured Analysis
 - A technique that helps the developer define what the system needs to do (the processing requirements), what data the system needs to store and use (data requirements), what inputs and outputs are needed, and how the functions work together overall to accomplish tasks.
 - Principle: focus on requirements, don't commit to specific technologies.
 - Models used in structured analysis
 - * Data flow diagram (DFD): A graphical model showing the inputs, processing, storage, outputs of a system produced in structured analysis.



- * Entity-relationship diagram (ERD): A graphical model of the data needed by a system, including things about which information is stored and the relationships among them, produced in structured analysis and information engineering.

BIT 2114 Object-Oriented Analysis and Design

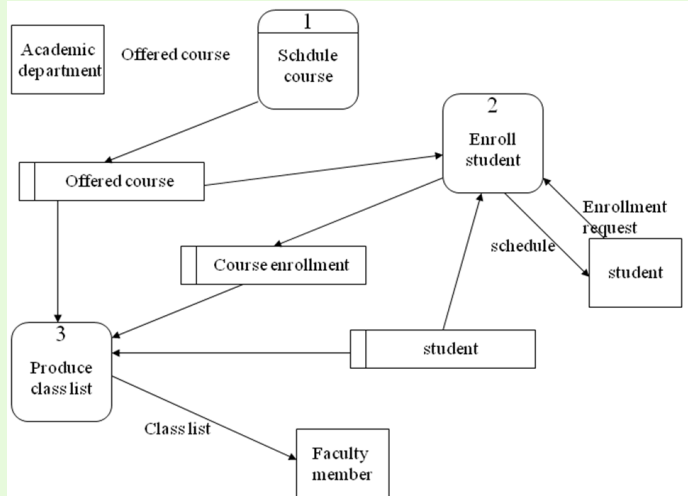


Figure 1.2: Sample DFD

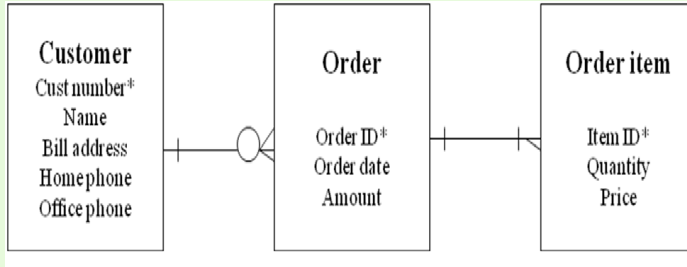


Figure 1.3: Sample ERD

Weakness of the Structured Approach

1. Address some, but not all, of the activities of analysis and design.
2. Techniques are not comprehensive and rigorous enough to make system development more like engineering discipline and less like an art.



BIT 2114 Object-Oriented Analysis and Design

3. Transition from Data flow diagram to structure chart is difficult.
4. Focus on process but data is more important (although SADT include data modeling and database design)
5. Need an overall strategic system plan

The Information Engineering (IE) Approach

A system development methodology that focus on strategic planning, data modeling, and automated tools. Thought to be more rigorous and complete than the structure approach.

Differences between SADT and the information engineering approach(IE):

1. IE begins with overall strategic planning



BIT 2114 Object-Oriented Analysis and Design

2. SADT focus on processes, IE focus on data
3. IE has a more complete life cycle and the use of CASE tools.
4. IE is more rigorous, but less flexible
5. Now Fewer companies use the IE approach exclusively
6. SADT's and IE's concepts are merged and be referred to as the traditional approach.

The Object-Oriented Approach

An approach to systems development radically different from the traditional approach, which views an information system as a collection of interacting objects that work together to accomplish tasks. Corresponding to Structured approach, object-



BIT 2114 Object-Oriented Analysis and Design

oriented approach consists of object-oriented analysis, object-oriented design, and object-oriented programming.

- Object
 - a thing in the computer system that can respond to messages.
- Object-oriented Analysis (OOA)
 - A technique that defines all of the types of objects that do the work in the system and shows how the objects interact to complete tasks.
- Object-oriented Design (OOD)
 - A technique that defines all of the types of objects necessary to communicate with people and devices in the system and refines the definition of each type



of object so it can be implemented with specific language or environment

- Object-oriented Programming (OOP)
 - Writing statements in a programming language to define what each type of object does, including the messages the objects send to each other.

1.1.3. Object Oriented Analysis and Design

Welcome to the world of object-oriented analysis and design. You are about to embark on learning a method which is fun, elegant, easy to understand, powerful and comprehensive.

The reasons for using object oriented design phase instead of going directly to implementation include:



BIT 2114 Object-Oriented Analysis and Design

1. The design phase deals with a higher level of abstraction than the analysis phase. Hence the system in is a finer detail form, enhancing its development.
2. The design phase exposes flaws that would not have otherwise been notices in the analysis phase.
3. Design techniques such as information hiding, abstraction, modularity, polymorphism etc, enhance the system's performance and development. These techniques are not much implemented in the analysis phase, if implemented at all.

Object modeling is useful for designing computer systems, whether those systems are to be implemented in object-oriented languages or not. Most designs are likely to need more than an object-oriented language, such as a database. Object modeling



BIT 2114 Object-Oriented Analysis and Design

also has a use outside of the design of computer systems. It is an excellent analysis method, and it can be used in business process re-engineering, in social science research, or any complex environment where it is important to capture the structure and functionality of some world.

Object-oriented programming is based on the principle of recursive design.

1. Everything is an object

Actions in OOP are performed by agents, called instances or objects. Each agent has a part to play, and the result is produced when all work together in the solution of a problem.

1. Objects perform computation by making requests of each other through the passing of messages

Actions in OOP are produced in response to requests for actions,



BIT 2114 Object-Oriented Analysis and Design

called messages. An instance may accept a message, and in return will perform an action and return a value.

Information Hiding

Notice that I, as a user of a service being provided by an object, need only know the name of the messages that the object will accept. I need not have any idea how the actions performed in response to my request will be carried out. Having accepted a message, an object is responsible for carrying it out.

Elements of OOP - Receivers

Messages differ from traditional function calls in two very important respects:

1. In a message there is a designated receiver that accepts the message



2. The interpretation of the message may be different, depending upon the receiver

Behavior and Interpretation

Although different objects may accept the same message, the actions (behavior) the object will perform will likely be different. The determination of what behavior to perform may be made at run-time, a form of late binding. The fact that the same name can mean two entirely different operations is one form of polymorphism, a topic we will discuss at length in subsequent chapters.

1. Every object has its own memory, which consists of other objects.

Each object is like a miniature computer itself - a specialized processor performing a specific task.



Non-interference

It is important that objects be allowed to perform their task however they see fit, without unnecessary interactions or interference with other objects.

1. Every object is an instance of a class. A class groups similar objects. The class is the repository for behavior associated with an object.

Behavior is associated with classes, not with individual instances. All objects that are instances of a class use the same method in response to similar messages.

Hierarchies of Categories

At each level of abstraction I have certain information recorded. That information is applicable to all lower (more specialized) levels.



BIT 2114 Object-Oriented Analysis and Design

1. Classes are organized into singly-rooted tree structure, called an inheritance hierarchy.

Information (data and/or behavior) I associate with one level of abstraction in a class hierarchy is automatically applicable to lower levels of the hierarchy.

- Inheritance promotes Re-use: When new objects are created which are similar to other objects, they can have many of their attributes and operations ready defined.
- Inheritance promotes clarity: The notion of breaking the world down into hierarchies of types is not as old as the hills, but it goes back at least as far as the Ancient Greeks: all men are mortal, Socrates is a man, so Socrates is mortal. The reductionist, classification approach is embedded in Western thought, and should be natural for most people



to follow.

Subclasses can alter or override information inherited from parent classes:

- All mammals give birth to live young
- A platypus is an egg-laying mammal

Inheritance combined with overriding are where most of the power of OO originates.

1.1.4. Computing as Simulation

The OOP view of computation is similar to creating a universe of interacting computing objects

- Similar to the way in which a committee or club might be organized



BIT 2114 Object-Oriented Analysis and Design

- Also very similar to a style of simulation called discrete event-driven simulation
- Easily underestimated advantage of this view – power of metaphor.

Metaphor and Problem Solving

Because the OOP view is similar to the way in which people go about solving problems in real life (finding another agent to do the real work!), intuition, ideas, and understanding from everyday experience can be brought to bear on computing. On the other hand, common sense was seldom useful when computers were viewed in the process-state model, since few people solve their everyday problems using pigeon-holes.

So why are Object Oriented concepts important?



BIT 2114 Object-Oriented Analysis and Design

1. Objects provide for modularity and encapsulation. Modularity implies that you can create units that can be used throughout a system or organization. Once an object is defined, this object definition and its properties can be used by other analysts and developers throughout the organization. Encapsulation or the ability to hide information means objects have public and private information, implying you can change the private information without affecting the other objects that use the public information.
2. Classes support the concept of reusability. Through inheritance, you have the basic information necessary to create additional objects without having to define the base information again.
3. Abstraction means objects are analyzed until they are uniquely



identified. Through these multiple analyzation iterations, you establish classes and subclasses. This iteration process supports the analytical process and helps to ensure a thorough analysis of the system.

4. Persistence means the classes and objects are permanent and will be available for use throughout the life of the system.

1.1.5. Criticisms of Object Oriented Analysis and Design

1. Lack of standardization. This criticism has been taken care of by the adoption of the Unified Modeling Language (UML) by the Object Management Group (OMG) in November 1997. UML was developed by three of the leading



proponents of object-oriented development, Grady Booch, James Rumbaugh and Ivar Jacobson, and provides a language for modelling systems. The OMG is an organization whose mission is to introduce standardization to object software; therefore, this adoption represented standardization in the analysis process.

2. The costly nature of adopting this paradigm. Change to OOAD requires the infamous paradigm shift, meaning you have to completely change your way of thinking and change your business processes as well as invest in training in order to ensure the staff is ready to accommodate the changes. This requires an investment in not only money but time.
3. Problems associated with the concept of reusability. It also must be kept in mind that software developers who



have used traditional methods for a long period of time tend to think in terms of process and, therefore, may find it difficult to change. The investment, however, may return dividends in the long run. A study comparing the data flow diagram method to the object-oriented analysis method showed that there was a longer learning curve associated with object-oriented analysis but, once learned, the object-oriented analysts performed better than the data flow diagram subjects in analyzing a system.

4. The applicability of adopting it for legacy systems. Finally, there is some question as the appropriateness of applying object-oriented techniques to legacy software. Object-oriented methodology starts with the object and this object drives the definition of further needs. With legacy



system, the data and its associated model already exist. You also have analysts steeped in traditional methodology more comfortable building the data model first. The good news is that once an analyst uses an object-oriented approach he is resistant to going back to traditional methods.

1.1.6. Benefits of Object-oriented Approach

1. Naturalness: People tend to think about the world in terms of tangible objects
2. Reuse: Classes can be used again and again whenever they are needed. There is less need to “reinvent the wheel”



1.2. Summary

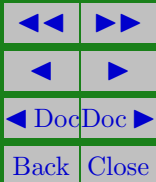
1. Object-oriented programming is not simply features added to a programming language. Rather, it is a new way of thinking.
2. Object-oriented programming views a program as a community of agents, termed objects. Each object is responsible for a specific task.
3. An object is an encapsulation of state (data values) and behavior (operations).
4. The behavior of objects is dictated by the object class. •
An object will exhibit its behavior by invoking a method (similar to executing a procedure) in response to a message.



JKUAT SODeL
©2014

BIT 2114 Object-Oriented Analysis and Design

5. Objects and classes extend the concept of abstract data types by adding the notion of inheritance.





Learning Activities

1. Describe two of the following objects in terms of their attributes and operations: motor car, sheep, kite, hospital, elephant, garden, school, bacon, teddy bear, bank customer, bus.
2. Devise a simple inheritance hierarchy for two of the following: geometry, sports teams, politicians, rodents, viral infections, restaurants.

Revision Questions

Example . Briefly explain the concept of an object and hence describe three characteristics of an object.

Solution: for revision





BIT 2114 Object-Oriented Analysis and Design

EXERCISE 1.  Briefly explain the UML

EXERCISE 2.  Discuss Object Oriented Principles and Concepts

References and Additional Reading Materials

1. Grady Booch (2007). Object oriented analysis and Design with applications, ISBN-13: 978-0201895513
2. J W. Satzinger, Robert B.Jackson, Stephen D. Burd (2004). Object oriented analysis and Design with Unified Process, ISBN-13: 978-0619216436